

Week 2 — ChatGPT Clone Project: Practice Exercises

Complete the ChatGPT clone project by following the Week 2 guided video sequence step by step. Recreate the backend in your own workspace, connect it to the provided frontend codebase, and verify that the full chat flow works from the browser.

Week goal

The goal for this week is to complete the ChatGPT clone project. By the end, the frontend should send a user's question to the backend, the backend should generate an AI response, save the conversation in the database, and return the result back to the chat UI.

The project should also use a system prompt so the assistant focuses only on programming, software engineering, computer science, and IT-related questions.

Complete the project by following all backend and frontend videos in order, then verify your final result using the completion criteria.

Hint on homework folder organization

- **Suggested folder layout:** Create a simple structure so everything for Phase 4 Week 2 stays in one place, for example:
 - `Evangadi/phase-4/week-2/chatgpt-clone/backend/` — your **Node / Express / MySQL backend**
 - `Evangadi/phase-4/week-2/chatgpt-clone/frontend/` — the **provided React frontend codebase**
 - Keep backend and frontend code inside the same `chatgpt-clone` folder so it is easy to run, test, and debug both sides.
 - Keep your `.env` file organized and do not share API keys publicly.
 - Do not skip directly to final code; code along while watching each video.
 - Resolve errors immediately before moving to the next section.
-

Build instructions

- In your course dashboard, open **Phase 4** → **Week 2**.
- Expand **Project Videos: ChatGPT Clone**.
- Start from the backend videos and continue in order.
- After completing the backend, use the provided frontend codebase and follow the frontend videos to connect and test the full application.

Backend videos

Watch and code along with these videos in order:

1. App Introduction and Goal
2. Initial Backend Setup
3. Database Setup
4. Folder Structure & Server Refactoring
5. Preparing the POST Request API
6. Preparing the GET Request API

Frontend videos

After the backend is ready, watch and code along with these videos in order:

1. Frontend Initialization & Backend Revision
2. Setup Sidebar Component
3. Setup Header Component
4. Setup Message List Component & Axios
5. Setup Individual Chat Component
6. Setup Chat Input Component

Frontend code setup instructions

- Download the provided frontend codebase:
[Download frontend codebase here](#)
- Unzip the downloaded file.
- Place the unzipped frontend project inside your Week 2 `chatgpt-clone` folder.
- Install dependencies in the frontend folder.
- Start the frontend development server.
- Make sure the frontend sends requests to your backend server.

Backend requirements

Your backend should include:

- A working Express server
- MySQL database connection
- `conversations` table for storing chat messages
- `GET /api/chat/conversations` API for loading saved chat history
- `POST /api/chat/conversations` API for sending a user question and receiving an assistant response
- Request validation for the `question` field
- AI model integration
- System prompt instructions that restrict the assistant to programming, software engineering, computer science, and IT-related questions
- Error handling for validation errors, database errors, and AI provider errors

API behavior requirements

By the end of the backend implementation:

- `GET /api/chat/conversations` should return saved chat messages from the database.
- `POST /api/chat/conversations` should save the user's question, call the AI model, save the assistant's answer, and return both saved messages.
- The assistant should politely reject unrelated questions and redirect the user back to programming topics.
- The frontend should be able to display old messages when the page loads.
- The frontend should be able to send a new question and display the assistant response.

Postman testing requirements

Before connecting the frontend, test the backend APIs with Postman:

- Test `GET /api/chat/conversations`
- Test `POST /api/chat/conversations`
- For the POST request, use **Body** → **raw** → **JSON** and send:

```
{  
  "question": "What is a closure in JavaScript?"  
}
```

Also test at least one unrelated question, for example:

```
{  
  "question": "What should I eat for dinner?"  
}
```

The assistant should not answer unrelated topics directly.

Completion checklist

By the end of Week 2, your project should demonstrate:

- A working ChatGPT clone backend
- Successful database connection and message storage
- Working `GET` and `POST` chat APIs
- Successful AI response generation
- System prompt behavior that keeps the assistant focused on programming-related questions
- Frontend connected to the backend
- Messages displayed correctly in the chat UI
- New user messages and assistant responses saved and shown correctly
- Clean project structure and runnable local setup

Happy coding 😊 .